

COMECE POR AQUI

# Guia rápido do V&V TestLab

Do primeiro cadastro ao relatório rastreável, com um exemplo completo e orientações para evitar as dúvidas mais comuns.

- |   |                                             |
|---|---------------------------------------------|
| 1 | Entenda o fluxo completo da plataforma      |
| 2 | Cadastre as três funcionalidades principais |
| 3 | Crie, execute e rastreie os casos           |
| 4 | Prepare a entrega e o relatório final       |

# Sumario

Leia este guia primeiro. Ele apresenta o fluxo essencial da plataforma e um exemplo que você pode adaptar ao seu projeto.

|                                      |          |
|--------------------------------------|----------|
| <b>1. O que você vai construir</b>   | <b>3</b> |
| <b>2. Iniciar em cinco minutos</b>   | <b>3</b> |
| <b>3. Mapa das telas</b>             | <b>3</b> |
| <b>4. Exemplo completo</b>           | <b>3</b> |
| <b>5. Criar um bom caso de teste</b> | <b>4</b> |
| <b>6. Executar e rastrear</b>        | <b>4</b> |
| <b>7. Automatizar com pytest</b>     | <b>4</b> |
| <b>8. Preparar a entrega</b>         | <b>5</b> |
| <b>9. Se algo não funcionar</b>      | <b>5</b> |

# 1. O que você vai construir

O V&V TestLab organiza o trabalho da disciplina em uma cadeia única e verificável:

```
Funcionalidade -> Classe de equivalência -> Caso de teste -> Execução -> Defeito
```

O escopo aceita exatamente as **três funcionalidades ou métodos mais importantes** do software analisado. A plataforma ajuda a demonstrar que cada requisito foi testado, executado e, quando necessário, associado a um defeito.

## 2. Iniciar em cinco minutos

No terminal aberto na pasta do V&V TestLab:

```
npm install
npm run dev
```

Abra <http://127.0.0.1:5173>. Use sempre esse endereço, pois `localhost` e `127.0.0.1` podem manter dados locais diferentes.

Em seguida:

- 1 Clique em **Configurar**.
- 2 Informe o software, o repositório e a meta de cobertura.
- 3 Abra **Requisitos** e cadastre as três funcionalidades.
- 4 Abra **Cenários e casos** e crie classes válidas e inválidas.
- 5 Crie ou gere casos, execute-os e confira **Rastreabilidade**.

## 3. Mapa das telas

| Tela                | O que fazer nela                                 |
|---------------------|--------------------------------------------------|
| Visão geral         | Conferir indicadores, etapa atual e pendências.  |
| Requisitos          | Manter as três funcionalidades principais.       |
| Cenários e casos    | Criar classes válidas/inválida, limites e casos. |
| Rastreabilidade     | Identificar elos ausentes entre os registros.    |
| Execução e métricas | Registrar resultados, cobertura e mutação.       |
| Integração Python   | Executar pytest e importar resultados.           |
| Defeitos            | Documentar falhas, evidências e correções.       |
| Dados e exportação  | Gerar PDF, CSV, Markdown e backup JSON.          |

## 4. Exemplo completo

Considere a funcionalidade **Cadastrar usuario** e a regra `18 <= idade <= 120`.

| Registro  | Exemplo                     |
|-----------|-----------------------------|
| Requisito | REQ-001 - Cadastrar usuario |

| Registro        | Exemplo                            |
|-----------------|------------------------------------|
| Classe válida   | CE-001 - Idade entre 18 e 120      |
| Classe inválida | CE-002 - Idade fora do intervalo   |
| Caso válido     | CT-001 - Aceitar idade 18          |
| Caso inválido   | CT-002 - Rejeitar idade 17         |
| Execução        | EXE-001 - CT-001 aprovado          |
| Defeito         | DEF-001 - Sistema aceitou idade 17 |

Na tela **Cenários e casos**, use **Gerar por limites** com mínimo 18 e máximo 120. A plataforma criará os pontos 17, 18, 19, 119, 120 e 121 e classificará os valores externos como inválidos.

## 5. Criar um bom caso de teste

Cada caso precisa responder cinco perguntas:

- 1 Qual funcionalidade esta sendo validada?
- 2 A qual classe válida ou inválida ele pertence?
- 3 Qual entrada será fornecida?
- 4 Quais passos precisam ser executados?
- 5 Qual resultado deve ser observado?

Exemplo:

| Campo               | Preenchimento sugerido                          |
|---------------------|-------------------------------------------------|
| Título              | Rejeitar cadastro com idade 17                  |
| Condição de entrada | Usuario novo com idade abaixo do mínimo         |
| Dados               | <code>{ "nome": "Ana", "idade": 17 }</code>     |
| Passos              | Preencher dados; confirmar cadastro             |
| Esperado            | Cadastro rejeitado e mensagem de idade inválida |
| Validade            | Invalido                                        |
| Técnica             | AVL                                             |

## 6. Executar e rastrear

Para uma execução manual, abra **Execução e métricas**, clique em **Nova execução**, selecione o caso e registre Aprovado, Falhou ou Bloqueado.

Depois abra **Rastreabilidade**. Uma cadeia completa deve ter requisito, classe, caso e execução. Falhas relevantes também devem possuir um defeito. Use o filtro **Com lacunas** para localizar rapidamente o que falta.

## 7. Automatizar com pytest

Associe o ID do caso ao teste Python:

```
import pytest

@pytest.mark.vv_case("CT-001")
def test_idade_minima():
    assert cadastrar_usuario(nome="Ana", idade=18).sucesso
```

Instale `pytest` e `pytest-cov`, inicie a ponte local e use **Executar e sincronizar**. O sistema importa execuções, cobertura, testes sem vínculo e defeitos gerados por falhas.

A execução automática precisa ser iniciada pela tela Integração Python. Testes executados separadamente pela IDE não são capturados nesta versão.

## 8. Preparar a entrega

Antes de gerar o PDF final do trabalho:

- Confirme as três funcionalidades.
- Confira classes válidas e inválidas.
- Execute todos os casos planejados.
- Registre cobertura e mutação.
- Documente defeitos e correções.
- Abra a matriz e resolva lacunas não justificadas.
- Gere um **Backup JSON**.
- Clique em **Gerar relatório PDF**.

## 9. Se algo não funcionar

| Problema                        | Ação recomendada                                           |
|---------------------------------|------------------------------------------------------------|
| Botão não responde              | Atualize com Ctrl+F5 e reinicie <code>npm run dev</code> . |
| Dados sumiram                   | Confira se abriu exatamente a mesma URL.                   |
| Quarta funcionalidade bloqueada | O limite acadêmico é três; edite ou substitua uma.         |
| Classe não aparece              | Selecione primeiro o requisito correto.                    |
| Ponte desconectada              | Confira terminal, URL, porta e ambiente Python.            |
| Teste não vinculado             | Use um ID existente como <code>CT-001</code> .             |
| PDF incompleto                  | Complete os registros e gere novamente.                    |